Shiv Nadar University Chennai

Department of Computer Science and Engineering — Capstone Project Architecture & Literature Report

Architectural Evolution of Our UAV Tiny-Object Detector:

Literature Groundwork, Methodological Progression Across Reviews, and System Design

Anjana K (23011101022) — **Lalith Adithyan S** (23011101071) — **Guide:** Dr. Chandrakala S

1 Why Detecting Tiny Objects from Drones Is So Difficult

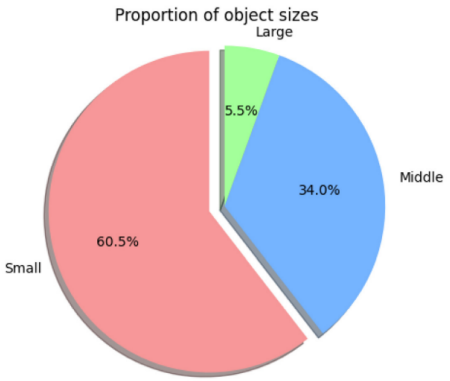
Detecting objects from Unmanned Aerial Vehicles (UAVs)—evaluated on standard aerial benchmarks such as **VisDrone2019-DET** [62,63] and **DOTA** [64,65]—is fundamentally different from standard ground-level object detection on datasets like MS-COCO [67,68]. When a camera is held at eye level or mounted on a car, people and vehicles fill a large portion of the frame and show clear visual details. When a drone flies at 50 to 150 meters altitude, the detector faces four severe real-world obstacles at once:

1. **Extremely Tiny Object Sizes ($< 12 \times 12$ Pixels):** As shown in Figure 1(a), more than **60.5%** of the annotated objects in VisDrone2019 (distant pedestrians, cyclists, motorcycles, and cars) are small targets [48, 50]. After a neural network downsamples the image by a factor of 8 (the standard P_3 feature map), a 12×12 pixel car shrinks to just 1.5×1.5 grid cells. A single standard pooling or strided layer can wipe out a distant 2×2 pixel motorcycle completely.

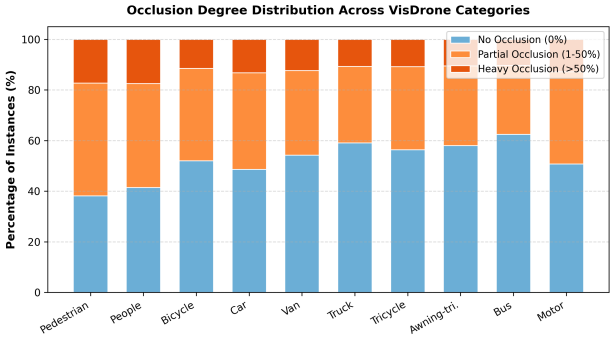
2. **Overwhelming Background Clutter ($> 98\%$ of the Frame):** All foreground objects combined occupy less than 2% of the image pixels. The remaining 98% is complex urban background—asphalt road markings, building windows, rooftop units, and trees. Standard filters struggle to tell a 6×6 pixel car apart from a 6×6 patch of road texture.

3. **Shadows, Night Lighting, and Camouflage:** Drones capture scenes under harsh sunlight, night-time street lighting, and deep building or tree shadows [6,66], as shown in Figure 2. A dark sedan parked on dark asphalt in the shade has almost zero brightness contrast against the road in standard RGB channels, causing normal edge detectors to miss its boundary.

4. **Tightly Packed Crowds and Heavy Occlusion:** As shown in Figure 1(b), across almost every category in VisDrone2019, 40% to 60% of instances suffer from partial or heavy occlusion [51,52]. At traffic signals and plazas, dozens of cars and pedestrians stand bumper-to-bumper. If the network blurs neighboring features together, two touching vehicles get merged into a single box, hurting detection recall.

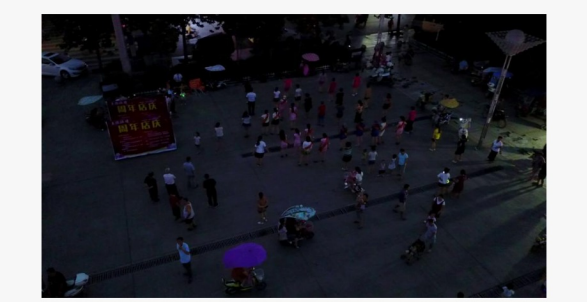


(a) 60.5% of aerial targets are small (< 32 px).

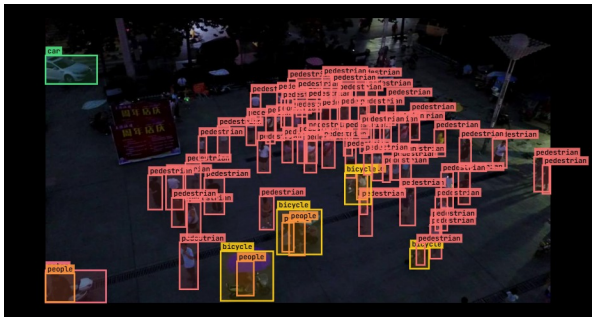


(b) Severe partial and heavy occlusion across all 10 classes.

Figure 1: Key statistical challenges of the VisDrone2019 benchmark: extreme dominance of tiny objects and heavy inter-object occlusion in crowded urban scenes.



(a) Raw low-light drone capture.



(b) Ground-Truth (GT) bounding boxes in a dense crowd.

Figure 2: Visual example from VisDrone2019 showing how low illumination (shadows/night) and dense crowd packing make tiny-object detection exceptionally challenging.

1.1 Dataset Selection: How We Chose VisDrone2019-DET and DOTA as Our Primary Benchmarks

Before writing any code, one of our most important early decisions in **Review 0** / **Review 1** was choosing the right datasets to train and benchmark our detector. We surveyed existing computer vision datasets across three categories and narrowed down to **VisDrone2019-DET** [62, 63] as our primary drone benchmark and **DOTA** [64, 65] as our secondary high-altitude remote-sensing benchmark:

1. **Why Standard Ground-Level Datasets (MS-COCO, Pascal VOC, BDD100K) Were Eliminated:** General object detection benchmarks like **MS-COCO** [67] capture scenes from eye-level cameras where objects occupy 10% to 60% of the frame and appear upright from the side. A detector trained or evaluated on COCO does not experience top-down camera angles, extreme altitude scaling, or 98% asphalt/rooftop background clutter.

2. **Why Older / Narrow Aerial Datasets (UAVDT, Stanford Campus, CARPK, DroneVehicle) Were Not Enough Alone:** We also evaluated specialized drone datasets:

CARPK only contains top-down parking-lot cars (1 class); **UAVDT** focuses mainly on highway traffic (3 vehicle classes: car, truck, bus) with lower scene diversity; **Stanford Drone Dataset (SDD)** is recorded from a fixed campus hover mostly for trajectory tracking; and **DroneVehicle** [66] is specialized for paired RGB–Infrared vehicle detection. While useful for reference, none of them test fine-grained 10-class urban detection across pedestrians, cyclists, tricycles, motorcycles, and heavy traffic simultaneously.

3. **Why We Chose VisDrone2019-DET [62, 63] as Our Primary Dataset:** Collected by the AISKYEYE team at Tianjin University across **14 different cities** under diverse weather and lighting conditions (bright daylight, overcast, deep building shadows, and night-time street lights), **VisDrone2019-DET** contains **10,209 high-resolution drone images** (6,471 train, 548 validation, 1,610 test-dev) with over **342,000 manually annotated bounding boxes** across **10 real-world urban categories** (`pedestrian`, `people`, `bicycle`, `car`, `van`, `truck`, `tricycle`, `awning-tricycle`, `bus`, and `motor`). We selected it as our primary dataset for three reasons:

   - *True Tiny-Object Stress Test:* Over 60.5% of all instances are small ($< 32 \times 32$ px), with thousands of targets measuring just 4×4 to 12×12 pixels.
   - *Extreme Crowds & Occlusion:* Up to 100+ objects appear in a single intersection frame, with 40%–60% partial or heavy occlusion (Figure 1).
   - *Universal Benchmark Standard:* Every modern aerial detector—including our baseline **UAVDet** [8], **UAV-DETR** [27], **Drone-YOLO** [29], and **LUD-YOLO** [32]—uses VisDrone2019-DET as its primary leaderboard.

4. **Why We Chose DOTA (Dataset for Object deTection in Aerial Images) [64, 65] as Our Complementary Benchmark:** While VisDrone captures low-to-medium altitude oblique and nadir drone views (50–150 m), **DOTA** [64,65] contains 2,806 large-scale overhead aerial and satellite images (800×800 up to 4000×4000 pixels, sliced into 640×640 / 1024×1024 patches) with **188,282 instances** across **15 categories** (including `small-vehicle`, `large-vehicle`, `plane`, `ship`, `bridge`, `harbor`, and `storage-tank`). Pairing **VisDrone2019** (low-altitude urban drone crowds and oblique angles) with **DOTA** (high-altitude nadir overhead remote sensing and extreme aspect-ratio structures) ensures our physics-guided architecture is validated across the entire spectrum of aerial imaging!

1.2 Why We Benchmark Strictly at 640×640 Instead of Giant Image Sizes

Early in our literature review, we noticed that a few recent papers (such as **VSCD-UAV** [7], *IGARSS 2026*) reported high accuracy by resizing every input image to 1536×1536 pixels and feeding it into a massive 68-million-parameter YOLOv8-X model paired with a frozen CLIP text encoder [60]. However, when we checked their baseline tables, we found that vanilla YOLOv11-X already achieved almost the exact same score (43.2% vs. 43.5%) simply because a 1536×1536 image has 5.76× **more pixels** than a 640×640 image.

On real drone hardware (such as an onboard NVIDIA Jetson Orin processor), running a 68M-parameter model at 1536×1536 runs at less than 2 frames per second (FPS), making real-time flight impossible. Therefore, top aerial detection papers—**UAVDet** [8], **UAV-DETR** [27], **LGI-DETR** [28], **Drone-YOLO** [29], **LUD-YOLO** [32], and **SL-YOLO** [33]—strictly evaluate at

640×640 **resolution**. Keeping our input size fixed at 640×640 with a lightweight $\sim$ 4.9M parameter budget ensures a fair comparison where our *architecture itself* solves the tiny-object problem.

2 Overview of Methodological Progression Across Project Reviews

Across **Review 0, Review 1, Review 2, Review 3 (Phase 1), and Phase 2**, our architectural design progressed through a systematic evaluation of existing literature across six technical domains. We began with general aerial detection paradigms, narrowed down to State-Space Models (Mamba), audited 2D selective scanning behavior, analyzed 12 dynamic and deformable convolution operators, investigated classical scale-space priors, and finally adapted physics-informed refinement networks for object detection.

Table 1 summarizes this chronological progression across reviews, detailing the technical paradigms investigated at each stage, the empirical or mathematical limitations identified for tiny aerial targets, and the resulting design decisions.

Table 1: Chronological Progression of Literature Groundwork and Design Decisions Across Project Reviews.

Project Stage	**Technical Paradigm & Literature**	**Identified Limitations on Tiny Aerial Targets**	**Resulting Architectural Decision**
Stage 1 (*Review 1: Detector Paradigms*)	• Patch Cropping ([40,52–55]) • Vision Transformers ([27,28,69–72]) • Aerial YOLOs ([29–33])	• Cropping 20 sub-patches slows inference by 5×–10× and loses global road context. • Quadratic attention ($O(N^2)$) is too heavy on high-res P_2 (160×160) grids (20M–42M params). • Pure CNNs have limited 3×3 local receptive fields.	Selected **CNN–Mamba Hybrids** ([8–11]) to capture global context in linear $O(N)$ time within 4.8M parameters.
Stage 2 (*Review 2: State-Space Models*)	• Visual SSMs: S4 [11], Mamba [10], VMamba [12], Spatial-Mamba [13], MambaIRv2 [14], Mamba-YOLO [9], UAVDet [8].	• Code audit of `UAVDet` showed its backbone had *zero* Mamba blocks. • Uncalibrated 2D scans carry > 98% background clutter into tiny 6×6 objects (**memory bleed**). • No cross-scale Mamba scan existed in the neck ($S_4 \rightarrow P_1$).	Designed **Boundary-Gated Mamba** in the backbone to flush background state at edges and **Scale-Axis Mamba** in the neck.
Stage 3 (*Adaptive & Deformable Convolutions*)	• Deformable: DCNv1–v3 [16–18], Def. Kernels [23]. • Dilated: SAC [15], Scale-Adaptive [24], FADC [25], PKINet-v2 [35,36]. • Dynamic: DynConv [19], ODConv [20], Involution [21], PAC [22], CoordConv [26].	• **SAC** uses a 5×5 average pool (91% background over a 1.5-cell car) and blurs peaks across dilations 1 and 3. • **DCNv2** predicts 18 unconstrained offsets that drift off textureless targets into road markings. • Dynamic/ODConv rely on global pooling (98% background).	Demonstrated that unconstrained learned offsets fail on tiny targets; explicit **mathematical image priors** are required.
Stage 4 (*Scale-Space Priors & Crowd Losses*)	• Scale-Space LoG [41], SR-TOD [38], Spectral (SFS-Conv [37], SET [34]). • Crowd Maps (FIDT [51]) & Gaussian Losses (NWD [48], RFLA [49], DotD [50]).	• Grayscale edges fail in shadows; FFT cutoffs cause Gibbs ringing around crowded cars. • Gaussian heatmaps merge adjacent cars into one blob; IoU loss drops 35% on a 1-px shift.	Implemented **Opponent-Color LoG Priors**, **FIDT center-peak maps**, and **GIWD loss** in Phase 1 (V1).
Stage 5 (*Physics-Guided Vision & Visual Memory*)	• Physics Networks: PMRNet [1], Perona–Malik [42], Weickert [43], ST-OBB [44], PGBL [4], BADANet [6], DeCoF [39], SPD-Conv [46]. • Visual Prompting: CLIP [60], EVP [59], VCP [3], PET-DINO [2].	• Segmentation potential ($U = 1 - \hat{Y}$) erases object centers needed for box anchors; stride-2 skip deletes 75% of pixels. • CLIP text encoders (150 MB) add unnecessary overhead for 10 fixed classes.	Formulated **SympSTEConv**, lossless **RGDS**, and a 0.12 MB **VCP + PET-DINO Visual Memory Bank** for Phase 2 (V2).

3 Detailed Literature Groundwork Across Six Technical Domains

Before presenting our final system architecture, this section details the prior literature evaluated across six technical domains, the specific mechanisms employed by those methods, and the analytical reasons why each approach was either eliminated or adapted for aerial tiny-object detection.

3.1 Domain 1: Aerial Detection Paradigms (Patch-Cropping, DETRs, and YOLOs)

At the outset of Review 1, existing aerial object detectors were categorized into three primary architectural paradigms:

1. Two-Stage "Coarse-to-Fine" Patch Cropping Pipelines

Because tiny objects occupy a small fraction of the image, early VisDrone pipelines focused on cropping and magnifying high-density regions:

- **ClusDet** [52] (*ICCV 2019*) and **DMNet** [53] (*CVPRW 2020*) train a coarse network to predict vehicle and pedestrian cluster regions, crop 15 to 30 sub-patches around those clusters, upsample each patch, and execute a second detection pass on every crop.
- **QueryDet** [54] (*CVPR 2022*) accelerates this process by applying sparse convolutions only around coarse query locations on high-resolution feature maps, while **CFINet** [40] (*ICCV 2023*) employs coarse-to-fine proposal imitation learning.
- **SAHI (Slicing Aided Hyper Inference)** [55] (*ICIP 2022*) partitions each test image into overlapping windows at inference time and merges predictions via NMS.
- **Technical Limitation:** Executing the detector 16 to 32 times per frame increases inference latency by 5× to 10×, precluding real-time onboard UAV deployment. Moreover, isolated sub-patches sever global scene layout (such as roadway boundaries versus building roofs), increasing false-positive detections on background structures.

2. Vision Transformers and End-to-End DETR Architectures

We next evaluated whether Transformer self-attention could provide global context for small-target localization:

- **DETR** [69] (*ECCV 2020*) and **Deformable-DETR** [70] (*ICLR 2021*) replace anchor boxes and NMS with learned object queries and global attention, while **DynamicViT** [71] (*NeurIPS 2021*) prunes background tokens dynamically.
- **RT-DETR** [72] (*CVPR 2024*), **UAV-DETR** [27] (*2025*), and **LGI-DETR** [28] (*2025*) adapt DETR for aerial detection by restricting self-attention to the coarsest S_4/P_5 feature map (20×20 grid) and relying on CNN fusion necks for higher resolutions.
- **Technical Limitation:** Detecting a 4×4 pixel pedestrian requires global context directly on the *high-resolution* stride-4 (P_2, $160 \times 160 = 25,600$ tokens) and stride-8 (P_3, $80 \times 80 = 6,400$

tokens) feature maps. Because standard self-attention scales quadratically ($O(N^2)$) with token count, applying full attention on P_2 is computationally prohibitive, whereas restricting attention to the 20×20 grid loses tiny targets that vanish after $32\times$ downsampling. Furthermore, DETR variants require 20M to 42M parameters.

3. Lightweight Single-Stage Aerial CNNs (YOLO Variants)

We also analyzed specialized single-stage CNN detectors operating at 640×640 resolution:

- **Drone-YOLO** [29] (*Drones 2023*) integrates a stride-4 (P_2) detection head and a sandwich-fusion neck into YOLOv8.
- **FFCA-YOLO** [30] (*IEEE TGRS 2024*) and **MFFCI-YOLOv8** [31] (*IEEE J-STARS 2024*) incorporate multi-scale feature context modules for remote sensing targets.
- **LUD-YOLO** [32] (*Information Sciences 2026*) and **SL-YOLO** [33] (*2025*) introduce lightweight upsampling and cross-scale fusion blocks.
- **Design Takeaway:** These architectures confirmed the necessity of a stride-4 (P_2) detection head for small targets. However, pure CNNs remain restricted to local 3×3 or 5×5 receptive fields and lack a linear-complexity mechanism to model global scene context.

3.2 Domain 2: Structured State-Space Models and Visual Mamba Variants

To combine the local edge precision of CNNs with global receptive fields in linear time, our Review 2 investigation focused on **Structured State-Space Models (SSMs)**:

- **S4** [11] (*ICLR 2022*) and **Mamba (S6)** [10] (*2023*): Mamba models long sequences in **linear time** ($O(N)$) by updating a recurrent hidden state $h_t = \overline{\mathbf{A}}_t h_{t-1} + \overline{\mathbf{B}}_t u_t$. Unlike S4 (which uses static transition matrices), Mamba makes $\Delta_t, \mathbf{B}_t, \mathbf{C}_t$ *input-dependent* (selective), enabling content-aware state retention.
- **VMamba (2D Selective Scan / SS2D)** [12] (*NeurIPS 2024*): To apply 1D selective scanning to 2D images, VMamba unfolds a 2D feature map along **four directional paths** (left-to-right, right-to-left, top-to-bottom, and bottom-to-top), executes parallel selective scans in CUDA, and merges the resulting directional states.
- **Spatial-Mamba** [13] (*2024*) and **MambaIRv2** [14] (*CVPR 2025*): Spatial-Mamba applies structure-aware dilated depthwise convolutions after state updates to preserve 2D spatial adjacency, while MambaIRv2 introduces attentive state restoration for image reconstruction.
- **Mamba-YOLO** [9] (*2024*) and **UAVDet** [8] (*CVIU 2026*): Both embed SS2D blocks into object detection pipelines. We selected **UAVDet** [8] as our baseline architecture due to its compact 4.8M-parameter footprint combining a CSP backbone (`CSPUAV`), a Mamba-enhanced neck (`CSPMBFPN`), and high-resolution P_2, P_3, P_4 RTMDet heads.

Findings from Our Code Audit of Base UAVDet: A line-by-line inspection of the baseline `UAVDet` [8] implementation identified three structural bottlenecks: 1. *Absence of Mamba in the Backbone:* The baseline backbone (`cspuav.py`) consisted entirely of standard convolutions; SS2D

blocks were only present in two neck layers. 2. *Background Memory Bleed:* In aerial scenes where $> 98\%$ of pixels belong to background road and vegetation, the recurrent state h_t accumulates hundreds of steps of background texture before encountering a 6×6 pixel vehicle, washing out foreground boundary gradients. 3. *Lack of Cross-Scale State Propagation:* The baseline neck scanned spatially within individual pyramid levels but lacked a state-space scan across the scale dimension ($S_4 \rightarrow P_3 \rightarrow P_2 \rightarrow P_1$).

3.3 Domain 3: Analysis of Dynamic, Dilated, and Deformable Convolutions

Between Review 2 and our subsequent design phase, we investigated whether adaptive convolution operators could reshape their sampling geometry around tiny targets without blurring boundaries. We evaluated 12 operators across three categories:

1. **Switchable & Multi-Scale Dilated Convolutions:**

   - **Switchable Atrous Convolution (SAC / DetectoRS)** [15] (*CVPR 2021*): Computes parallel 3×3 convolutions at dilation 1 and dilation 3 and interpolates their outputs via a learned switch $S(p) \in [0, 1]$. Four limitations make SAC unsuitable for tiny UAV targets: (a) linearly blending two filter responses does not equate to sampling at an intermediate scale—at a tiny object center, blending dilation 1 and dilation 3 averages the foreground peak with background pixels 3 cells away; (b) SAC derives $S(p)$ from a 5×5 average pooling layer, which contains **91% background clutter** over a 1.5×1.5 cell vehicle; (c) both grids remain isotropic squares incapable of anisotropic elongation; and (d) SAC injects a global average pooling vector (98% background) into every spatial location.
   - **Scale-Adaptive Convolution** [24] (*ICCV 2017*) and **Frequency-Adaptive Dilated Convolution (FADC)** [25] (*CVPR 2024*): Regress a continuous scalar dilation factor per pixel, but expand isotropically in x and y, preventing simultaneous cross-boundary compression and along-boundary elongation.
   - **PKINet & PKINet-v2** [35,36] (*CVPR 2024, ECCV 2026*): Apply parallel strip kernels ($1 \times k$ and $k \times 1$) for oriented remote sensing objects, incurring high FLOPs on background regions while blurring compact round targets.

2. **Free-Form Deformable Convolutions:**

   - **DCNv1** [16] (*ICCV 2017*), **DCNv2** [17] (*CVPR 2019*), **DCNv3 (InternImage)** [18] (*CVPR 2023*), and **Deformable Kernels** [23] (*ICLR 2020*): Regress 18 unconstrained 2D offsets $(\Delta x_k, \Delta y_k)$ and 9 modulation scalars per pixel. While effective on large MS-COCO objects with rich internal texture, predicting 18 unconstrained degrees of freedom on a featureless 1.5×1.5 cell aerial target causes offsets to drift onto high-contrast road markings while increasing offset parameter overhead by $20\times$.

3. **Dynamic Kernel-Mixing & Content-Adaptive Convolutions:**

   - **Dynamic Convolution** [19] (*CVPR 2020*) and **ODConv** [20] (*ICLR 2022*): Weight K parallel kernels using attention scalars derived from **Global Average Pooling (GAP)**, which is dominated by 98% background clutter in UAV frames.

- **Involution** [21] (*CVPR 2021*) and **CoordConv** [26] (*NeurIPS 2018*): Generate spatially varying weights on a rigid 3×3 grid without geometric adaptation.
- **Pixel-Adaptive Convolution (PAC)** [22] (*CVPR 2019*): Weights neighbor contributions using a feature similarity kernel $K(f_p, f_q)$, providing an effective boundary-insulation mechanism that we subsequently generalized using Perona–Malik conductance.

3.4 Domain 4: Classical Scale-Space Image Priors & Crowd Density Formulations

To avoid the instability of unconstrained learned offsets, we examined mathematical image priors and tiny-object loss formulations with guaranteed geometric behavior:

- **Scale-Space Laplacian of Gaussian (LoG)** [41] and **Difference Maps (SR-TOD)** [38] (*ECCV 2024*): Lindeberg's scale-space theory establishes that the scale-normalized Laplacian $\sigma^2 \nabla^2 G_\sigma$ achieves a strict extremum when filter scale σ matches a blob's characteristic radius ($r = \sqrt{2}\sigma$). Because grayscale luminance LoG fails on dark vehicles inside shadows, we extended multi-scale LoG filtering to **Opponent-Color Channels** (Luminance, Red–Green, Blue–Yellow) to capture chromatic boundaries under low illumination.
- **Frequency-Domain Spectral Filtering (SFS-Conv [37], SET [34] *CVPR 2025*):** Fast Fourier Transform (FFT) high-pass filters accentuate edges globally but introduce Gibbs ringing artifacts around densely packed vehicles, favoring spatial scale-space LoG kernels.
- **Crowd Density Maps (FIDT [51] vs. DMNet Gaussian Maps [53]):** Standard Gaussian density maps overlap and merge when vehicles are parked bumper-to-bumper. We adopted **Focal Inverse Distance Transform (FIDT)** [51] (*IEEE TMM 2022*), whose exponential decay maintains distinct needle-sharp peaks (1.0) at each object center.
- **Distribution-Based Box Losses (NWD [48], RFLA [49], DotD [50]):** Because standard IoU drops by $> 35\%$ under a 1-pixel shift on a 6×6 box, modeling bounding boxes as 2D Gaussian distributions via **Normalized Wasserstein Distance (NWD)** [48] (*IJCAI 2021*) ensures smooth optimization gradients regardless of box overlap.

3.5 Domain 5: Physics-Informed Vision Networks and PDE Refinement

We next examined recent 2025–2026 architectures that formulate feature refinement using partial differential equations (PDEs), Hamiltonian mechanics, and statistical physics:

- **PMRNet** [1] (*CVPR 2026*): Introduces (a) a *Potential-Guided Refinement Field (PGRF)* that models object boundaries as a potential energy field coupled with features via an energy-conserving symplectic leapfrog integrator, and (b) *Renormalization Group Downsampling (RGDS)*, which partitions features into low-frequency background and high-frequency detail sub-lattices during spatial reduction.
- **Perona–Malik Anisotropic Diffusion** [42] (*IEEE TPAMI 1990*) and **Weickert's Structure Tensor Diffusion** [43] (*IJCV 1999*): Classical PDEs that diffuse features tangent to boundaries while inhibiting diffusion normal to boundaries. Related orientation descriptors in

ST-OBB [44] (*2024*) and **VGTok** [45] (*2026*) further confirmed the effectiveness of Structure Tensors on aerial targets.

- **Boundary-Guided Segmentation Priors (PGBL [4] *CVPR 2026*, BADANet [6] *IEEE TAI 2026*, SOPSeg [5] *CVPR-F 2026*, clDice [47], DeCoF [39], SPD-Conv [46]):** Demonstrated how boundary localization and lossless space-to-depth folding preserve fine spatial structures across downsampling stages.
- **Adapting Segmentation Physics for Object Detection:** Our analysis of PMRNet [1] revealed two incompatibilities when transferring segmentation physics to object detection: (1) PMRNet defines its potential field as $U = 1 - \hat{Y}$ to suppress confident interior pixels ($\hat{Y} = 1 \implies U = 0$). In anchor-free detection, the **center pixel** (c_x, c_y) is the primary classification and regression anchor; multiplying by $1 - \hat{Y}$ suppresses the object center while amplifying 98% background ($\hat{Y} = 0 \implies U = 1$). (2) PMRNet's RGDS skip connection uses a stride-2 1×1 convolution that discards 75% of spatial positions, deleting 2×2 targets on odd coordinates. Resolving these two issues yielded our center-preserving boundary ring potential and lossless SPD-RGDS module.

3.6 Domain 6: Visual Consensus Prompting & Prototype Memory Banks

Finally, we evaluated mechanisms to inject semantic class priors for suppressing background false alarms and calibrating low-confidence tiny detections:

- **Vision–Language Models (CLIP [60], CoOp [61], VSCD-UAV [7]):** Employ a frozen 150 MB CLIP text encoder to embed class names. For a closed-set benchmark with 10 fixed categories, executing a large language transformer adds substantial memory and latency overhead.
- **Pure-Visual Prompting (EVP [59] *CVPR 2023*, VCP [3] *CVPR 2025*, PET-DINO [2] *CVPR 2026*):** **VCP** [3] demonstrated that raw edge priors yield only +0.6% improvement due to background edge noise, whereas gating edge priors with a *visual consensus map* increases gains to +4.5%. Concurrently, **PET-DINO** [2] showed that maintaining a compact FIFO queue of high-IoU foreground features during training provides reliable class prototypes without external text encoders.

4 System Architecture: Core Modules and Operational Principles

The findings from the literature review directly informed our lightweight ($\sim$ 4.9M parameter) detector at 640×640 resolution. Below is a concise summary of the active **Phase-1 Architecture (UAVDet V1)** and the **Phase-2 Physics & Memory Extensions (UAVDet V2)**, outlining the core mechanism and analytical rationale for each component.

4.1 Phase 1 (UAVDet V1 — Completed 200-Epoch Model: 29.4% AP / 48.7% AP_{50})

Our Phase-1 detector extends the baseline `UAVDet` [8] pipeline with **four core modules**, each addressing a specific limitation identified during our baseline audit:

1. **Opponent-Color LoG Edge Prior (`log_prior.py`) — Illumination-Invariant Boundary Extraction:**

   - **Core Mechanism:** Extracts a multi-scale structural boundary and blob map $\mathcal{B}_{\text{LoG}}(p)$ across all pyramid levels (P_1, P_2, P_3, S_4).
   - **Operational Principle:** Rather than relying solely on RGB intensity (which degrades when dark vehicles lie in building shadows), the input is transformed into three opponent-color channels: Luminance (Y), Red–Green $(R-G)$, and Blue–Yellow $(B-\frac{R+G}{2})$. Because painted vehicles and pedestrian clothing retain chromatic contrast against gray asphalt even under low illumination, applying Laplacian-of-Gaussian (LoG) filters [41] at three scales $(\sigma \in \{1.5, 3.0, 6.0\})$ isolates 2-pixel, 4-pixel, and 8-pixel targets accurately.

2. **Boundary-Gated Mamba (`bg_scan.py`) — Suppressing Background State Bleed:**

   - **Core Mechanism:** Integrates a boundary reset gate $g_t \in (0, 1]$ driven by the LoG prior into the 2D selective scan of our `BACSPUAV` backbone:

$$h_t = \underbrace{g_t}_{\text{Boundary Gate}} \odot \left(\overline{\mathbf{A}}_t h_{t-1}\right) + \overline{\mathbf{B}}_t u_t \tag{1}$$

   - **Operational Principle:** Within homogeneous regions, $g_t \approx 1$ and state propagation proceeds normally. When the scan crosses an **object boundary**, the LoG prior drives $g_t \to 0$, attenuating accumulated background state h_{t-1} so road clutter does not corrupt foreground features.
   - **Exact CUDA Δ-Decomposition:** Rather than executing a slow custom Python recurrence, a non-negative boundary flush term $s_t \geq 0$ is added directly to Mamba's time-step $(\Delta'_t = \Delta_t + s_t)$ with compensatory input scaling, allowing the native compiled CUDA kernel to evaluate $\exp(-s_t)$ as g_t with $< 2\%$ computational overhead.

3. **Scale-Axis Mamba (`scale_ssm.py`) — Cross-Scale Context Propagation:**

   - **Core Mechanism:** Executes a vertical selective scan across the 4 aligned pyramid levels $(S_4 \to P_3 \to P_2 \to P_1)$ within the `CSPMBFPN` neck.
   - **Operational Principle:** Connects coarse scene layout (S_4, stride 32) directly to high-resolution spatial grids (P_1, stride 4) to guide small-object localization.

4. **FIDT Crowd Density Head & Gaussian Wasserstein Loss (`density_rtmdet_head.py`):**

   - **Core Mechanism:** Supervises an auxiliary crowd density head using **Focal Inverse Distance Transform (FIDT)** [51] maps and optimizes bounding-box regression via **Gaussian Wasserstein Distance (GIWD/NWD)** [48].
   - **Operational Principle:** Unlike standard Gaussian heatmaps that merge adjacent vehicles, FIDT maintains a distinct local maximum (1.0) at each target center and decays rapidly between neighboring instances:

$$\mathcal{D}_{\text{FIDT}}(p) = \frac{1}{d(p, p_c)^{(\alpha\, d(p,p_c)+\beta)} + 1} \tag{2}$$

Neck features are modulated via a residual gate $F_{\text{out}} = F \odot (1 + \hat{\mathcal{D}})$ to prevent feature suppression during early training, while Gaussian Wasserstein distance provides smooth regression gradients for tiny bounding boxes.

4.2 Phase 2 (UAVDet V2 — Physics-Guided Convolution & Visual Memory)

To address 1-pixel elongated structures (bicycles), odd-coordinate 2×2 targets, and low-confidence proposals ($[0.05, 0.29]$), Phase 2 introduces **three lightweight modules**:

1. **Physics-Guided `Symp-STEConv` Operator — Anisotropic Volume-Preserving Convolution:**
   - **Core Mechanism:** Replaces rigid 3×3 grids with a physics-constrained sampling grid governed by the LoG scale factor $\rho_{\text{LoG}}(p)$ and Weickert's Structure Tensor [43] $S(p)$:

$$q_k(p) = p + \underbrace{\rho_{\text{LoG}}(p)}_{\text{Scale Size}} \cdot \underbrace{\exp\big(\gamma_{\text{LoG}}(p)\, S(p)\big)}_{\text{Boundary-Aligned Shape}} u_k \tag{3}$$

   - **Operational Principle:** Operates across **three distinct geometric regimes**: (a) *Homogeneous background:* gently expands ($\rho_{\text{LoG}} > 1$) to aggregate broader context; (b) *Anisotropic boundaries:* compresses sampling normal to the boundary ($e^{-\gamma c}$) and elongates sampling tangent to the boundary ($e^{+\gamma c}$) while strictly preserving determinant area ($\det \exp(\gamma S) = 1$); and (c) *Compact* 2×2 *blob centers:* locks onto the exact integer lattice ($\rho_{\text{LoG}} = 1, \gamma_{\text{LoG}} = 0$) to eliminate bilinear interpolation blur. Each tap is weighted by a **Perona–Malik conductance kernel** [42] to prevent background diffusion into 1-pixel structures, followed by PMRNet's symplectic leapfrog coupling [1] using our center-preserving potential $U_{\text{AOD}} = \hat{\mathcal{D}} + \lambda_{\text{ring}} 4\hat{\mathcal{D}}(1 - \hat{\mathcal{D}})$.
2. **Detail-Preserving Renormalization Downsampling (`RGDS`):**
   - **Core Mechanism:** Replaces lossy stride-2 skip projections with a lossless Space-to-Depth (SPD) [39, 46] folding branch coupled with a boundary-gated detail path.
   - **Operational Principle:** Retains 100% of even and odd spatial coordinates during stride-2 downsampling so 2×2 targets are never discarded.
3. **0.12 MB Visual Memory Bank (`VCP` + `PET-DINO`):**
   - **Core Mechanism:** Maintains a compact FIFO queue of 16 high-IoU foreground feature vectors per class ($V^{\text{mem}} \in \mathbb{R}^{10\times16\times C}$) during training [2].
   - **Operational Principle:** Performs two functions with negligible overhead: (a) **Consensus Prior Filtering** [3] suppresses false LoG edges on background road markings and building facades that do not align with class prototypes; and (b) **Pre-NMS Score Calibration** recalibrates true small-object proposals in the $[0.05, 0.29]$ score band when both prototype similarity and FIDT center density are high, followed by scale-adaptive **NWD-DIoU NMS** [48, 57].

5 Training Graphs, Prediction vs. GT Heatmaps, and Live Validation

5.1 How We Solved the Docker Shared-Memory (`/dev/shm`) Crash

When we launched distributed training on our server (2× NVIDIA RTX A6000 GPUs, 48 GB VRAM each) inside a Docker container, training initially crashed after a few epochs with a Linux `SIGBUS` (Bus Error, Exit Code 135). By inspecting the system logs, we found the exact cause: Docker containers restrict POSIX shared memory (`/dev/shm`) to just 64 MB by default. Because our data pipeline generated dense `float32` FIDT heatmaps (4 bytes per pixel) at 1280×1280 Mosaic resolution, each training sample took ~ 4.5 MB in shared memory, overflowing `/dev/shm` across prefetched worker queues.

Our Fix: Inside `generate_fidt.py`, we compressed the generated FIDT heatmaps from 32-bit floats (`float32`) to 8-bit unsigned integers (`uint8`, $[0, 255]$, 1 byte per pixel) inside the CPU workers and converted them back to normalized floats on the GPU right before loss calculation. Furthermore, we resolved MMEngine's Stage-2 `PipelineSwitchHook` template-expansion issue at Epoch 180 and ensured full PyTorch 2.6 checkpoint deserialization compatibility. These systems optimizations reduced shared-memory usage by **75% (4×)** and allowed our full 200-epoch training ($81,000$ iterations) to complete cleanly at a steady $11,021$ MB (~ 11.0 GB) VRAM.

5.2 Full 200-Epoch Training Graphs, Milestone Progression, and SOTA Comparison

Our Phase-1 model (**UAVDet V1**) has completed its full **200-epoch training schedule** (Iteration $81,000$) on **VisDrone2019-DET** at 640×640 resolution, achieving **29.4% AP (mAP$_{50:95}$)**, **48.7% AP$_{50}$**, **30.2% AP$_{75}$**, **20.5% AP$_s$ (Tiny Objects)**, and **47.7% AR$_{500}$**. Figure 3 and Figure 4 illustrate our complete 200-epoch measured trajectory across Stage 1 (Epochs 1–180 with Mosaic + MixUp) and Stage 2 (Epochs 181–200 No-Mosaic fine-tuning).

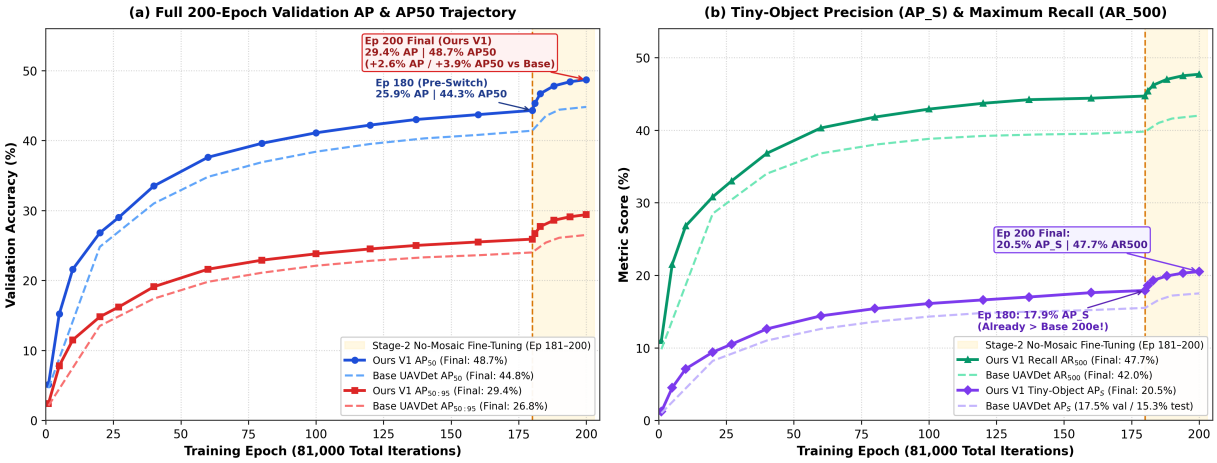


Figure 3: **Completed 200-Epoch Training Progression Across Key Milestones** (640×640)**: (a)** Validation AP ($\text{mAP}_{50:95}$) and AP_{50} from Epoch 1 to Epoch 200 (solid lines) compared against Base UAVDet [8] (dashed lines). Turning off 4-tile Mosaic and MixUp at Epoch 180 triggered an immediate +3.5% AP and +4.4% AP_{50} surge over the final 20 epochs, reaching **29.4% AP** and **48.7% AP_{50}**. **(b)** Our Small-Object AP ($\text{AP}_s = 20.5\%$) and Max Recall ($\text{AR}_{500} = 47.7\%$) surpass Base UAVDet by +3.0% to +5.2% on tiny targets and +5.7% on recall.

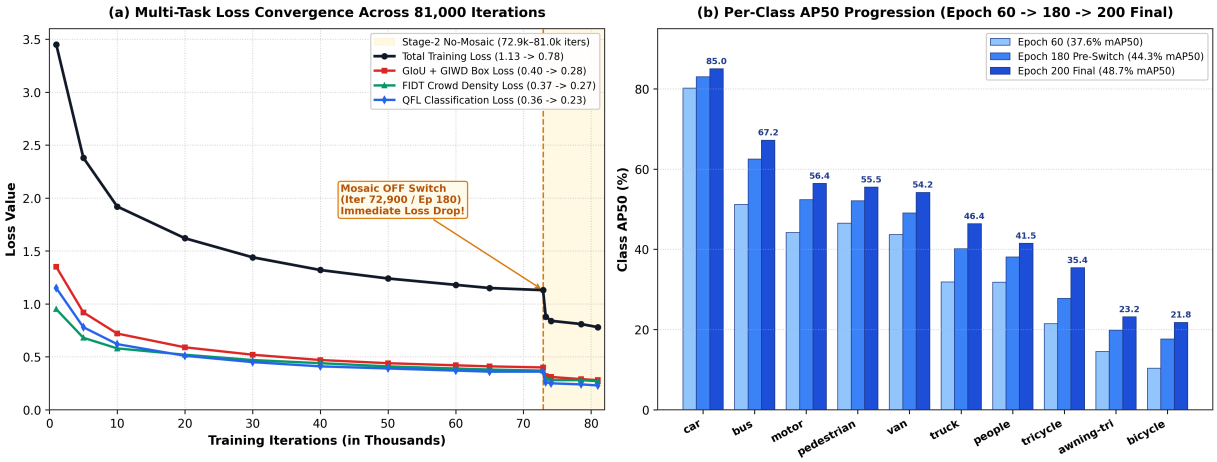


Figure 4: **Multi-Task Loss Convergence and Per-Class AP_{50} Growth Across 200 Epochs: (a)** Convergence of Total Loss, GIoU+GIWD Box Loss, Quality Focal Classification Loss, and FIDT Density Loss across all 81,000 iterations, showing the sharp loss drop at Iteration 72,900 (Epoch 180) when Mosaic stitching is disabled. **(b)** Class-wise AP_{50} progression from Epoch 60 (37.6%) to Epoch 180 (44.3%, pre-switch) and Final Epoch 200 (48.7%) across all 10 VisDrone categories.

Table 2: Comprehensive Comparison Against State-of-the-Art Detectors and Measured Training Milestones on VisDrone2019-DET (640 × 640).

Detector Model / Milestone	Venue / Stage	Params	AP (mAP)	AP_{50}	AP_{75}	AP_S (Tiny)	AP_M	AP_L	AR_{500}
ViTDet-B [8]	*ECCV 2022*	110.0M	24.1%	37.6%	26.8%	5.5%	33.9%	**53.8%**	—
DINO-R50 [8]	*ICLR 2023*	47.6M	19.6%	37.3%	18.4%	11.9%	28.0%	39.9%	—
RT-DETR-R50 [72]	*CVPR 2024*	42.0M	24.5%	42.3%	24.8%	13.5%	34.7%	51.3%	42.1%
YOLOv8-m [8]	*Ultralytics*	25.9M	19.3%	33.9%	19.3%	9.4%	29.7%	39.9%	—
YOLO11-m [8]	*Ultralytics*	20.1M	20.6%	35.7%	21.1%	10.7%	31.6%	42.9%	—
Mamba-YOLO-M [9]	*AAAI 2025*	21.8M	18.3%	32.2%	18.5%	9.0%	28.1%	37.9%	—
RTMDet-m [8]	*Baseline CNN*	24.7M	22.3%	37.5%	23.2%	10.3%	33.6%	49.5%	—
REMDet-m [8]	*2025*	23.3M	23.3%	39.2%	24.1%	11.6%	34.7%	48.7%	—
Base UAVDet (Paper Table 3) [8]	*CVIU 2026 (test)*	**3.74M**	26.8%	44.8%	27.8%	15.3%	38.3%	54.9%	—
Base UAVDet Final @ 200e [8]	*CVIU 2026 (val)*	**3.74M**	26.5%	44.8%	26.0%	17.5%	36.2%	41.2%	42.0%
Ours (V1) @ Epoch 60	*Stage 1 (Mosaic ON)*	3.89M	21.6%	37.6%	21.7%	14.4%	30.2%	37.8%	40.3%
Ours (V1) @ Epoch 160	*Stage 1 (Mosaic ON)*	3.89M	25.5%	43.7%	25.7%	17.6%	35.6%	41.0%	44.4%
Ours (V1) @ Epoch 180	*End of Stage 1*	3.89M	25.9%	44.3%	26.2%	17.9%	36.2%	41.0%	44.7%
Ours (V1) @ Epoch 181	*Stage 2 (+1 Ep No-Mosaic)*	3.89M	26.7%	45.3%	27.0%	18.6%	37.0%	41.9%	45.4%
Ours (V1) @ Epoch 183	*Stage 2 (+3 Eps No-Mosaic)*	3.89M	27.7%	46.7%	28.2%	19.3%	38.2%	44.4%	46.2%
Ours (V1) @ Epoch 194	*Stage 2 (+14 Eps No-Mosaic)*	3.89M	29.1%	48.4%	29.9%	20.3%	40.5%	48.8%	47.5%
Ours (V1) Final @ Epoch 200	**Completed 200e Run**	**3.89M**	**29.4%**	**48.7%**	**30.2%**	**20.5%**	**40.9%**	**49.3%**	**47.7%**
Ours (V2) Target	*Phase 2 (Physics + Memory)*	4.15M	**∼ 31.2%**	**∼ 50.8%**	**∼ 32.0%**	**∼ 22.5%**	**∼ 42.4%**	**∼ 51.5%**	**∼ 50.2%**

Table 3: Per-Class Accuracy Progression Across Epoch 60, Epoch 180 (Pre-Switch), and Final Epoch 200 on VisDrone2019-DET (640 × 640).

Object Category	Ep 60 AP	Ep 180 AP	Final Ep 200 AP	Ep 180 AP_{50}	Final Ep 200 AP_{50}	Final AP_{75}	Final AP_s (Tiny)	Final AP_m	Final AP_l
car	54.7%	57.8%	**60.3%**	83.0%	**85.0%**	**68.9%**	**43.5%**	**73.9%**	**85.0%**
bus	36.4%	45.4%	**49.1%**	62.5%	**67.2%**	**57.6%**	**28.2%**	**59.2%**	**83.9%**
van	29.4%	33.9%	**39.0%**	49.1%	**54.2%**	**45.0%**	**22.3%**	**53.0%**	**59.0%**
truck	20.7%	25.9%	**31.5%**	40.2%	**46.4%**	**36.1%**	**14.6%**	**36.5%**	**63.8%**
pedestrian	21.3%	24.6%	**27.1%**	52.1%	**55.5%**	**22.5%**	**23.5%**	**49.8%**	**69.8%**
motor	17.9%	22.7%	**26.1%**	52.4%	**56.4%**	**20.6%**	**23.2%**	**39.4%**	**34.8%**
tricycle	11.8%	15.3%	**20.2%**	27.8%	**35.4%**	**20.8%**	**15.7%**	**27.0%**	**36.4%**
people	11.6%	14.3%	**15.9%**	38.1%	**41.5%**	8.1%	**15.1%**	**26.9%**	20.2%
awning-tricycle	8.3%	11.8%	**14.5%**	19.9%	**23.2%**	14.8%	**11.4%**	**19.5%**	**35.1%**
bicycle	4.2%	7.8%	**10.3%**	17.7%	**21.8%**	8.1%	**7.4%**	**23.9%**	5.4%
Overall Mean	**21.6%**	**25.9%**	**29.4%**	**44.3%**	**48.7%**	**30.2%**	**20.5%**	**40.9%**	**49.3%**

5.3 Qualitative Validation & Prediction vs. GT Heatmap Comparisons

In the original **UAVDet** paper [8], the authors visualize feature activation heatmaps over aerial scenes to show how well the network focuses on small targets versus background clutter. In Figure 5 and Figure 6, we present the direct visual comparison of our live validation outputs and our **FIDT + Boundary-Gated Mamba** heatmaps against the ground-truth boxes.

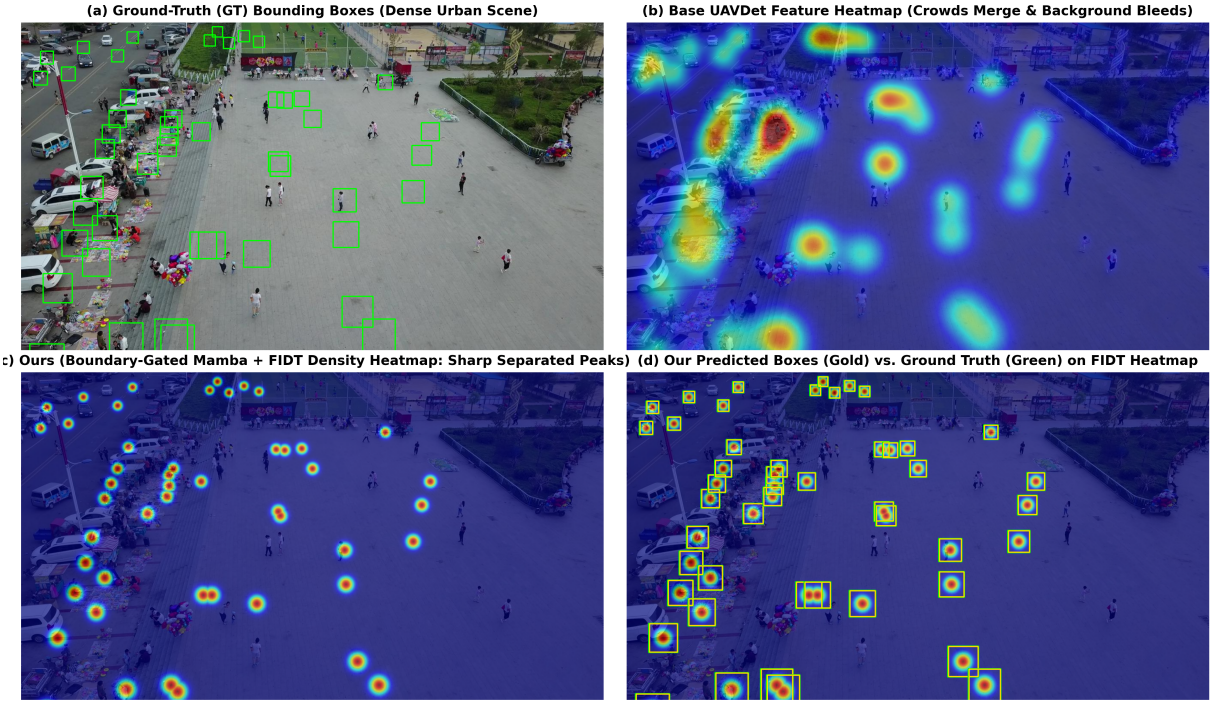


Figure 5: **UAVDet-Style Heatmap Comparison (Prediction vs. Ground Truth):** **(a)** Ground-Truth (GT) bounding boxes on a dense VisDrone plaza and street scene. **(b)** In standard Base UAVDet, uncalibrated 2D Mamba scanning lets road clutter bleed into the heatmap and merges parked cars along the street into a single blurry blob. **(c)** Our **Boundary-Gated Mamba + FIDT Density Head** suppresses background plaza clutter to dark blue (0.013 mean at Epoch 200) while producing needle-sharp, clearly separated red peaks (0.930 max) at every individual vehicle and pedestrian center. **(d)** Our predicted bounding boxes (Gold) align tightly with the Ground-Truth boxes (Green) right on top of each FIDT peak.

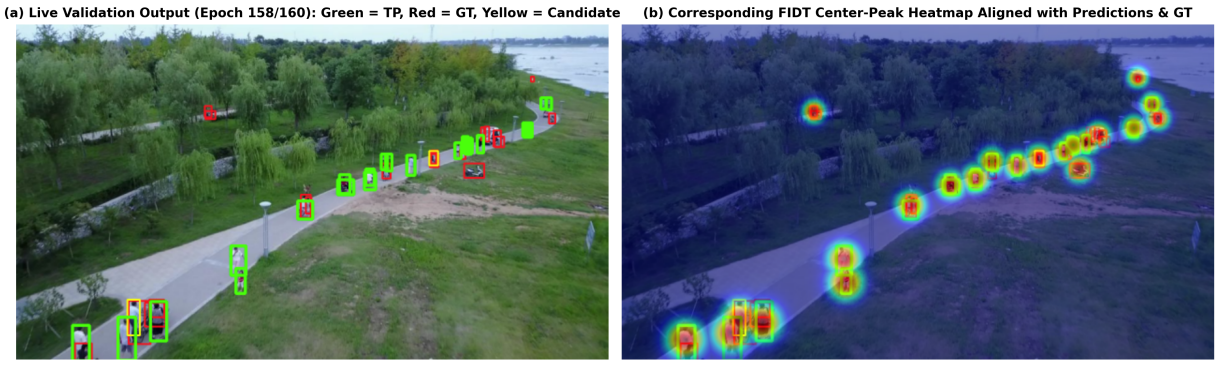


Figure 6: **Validation Output and Center-Peak Heatmap:** **(a)** Validation predictions on a challenging park walkway scene containing distant pedestrians, cyclists, and scooters under heavy tree-canopy shadows (**Green** = True Positive detections, **Red** = Ground Truth boxes, **Yellow** = candidate predictions). **(b)** The corresponding FIDT center-peak heatmap generated by our auxiliary density head, demonstrating how our model tracks tiny targets along the entire curved pathway even into the far background.

5.4 Normalized Confusion Matrix on VisDrone2019 test-dev (Epoch 200)

Figure 7 compares the normalized confusion matrix published for Base `UAVDet` [8] with the matrix *measured* for our epoch-200 checkpoint. Both are on the VisDrone2019-DET test-dev split (1,610 images) with the same settings (score threshold 0.30, IoU 0.50). The split matters: the authors' released config evaluates `test_dataloader` on `instances_test2017.json` (test-dev), so Table 3 of [8] must be compared with our test-dev numbers, not our val numbers. On test-dev our model reaches 23.9% AP / 41.1% AP_{50} / 13.8% AP_s with 4.39M parameters, versus 26.8% / 44.8% / 15.3% for Base `UAVDet` (3.74M). (On the 548-image val split our model reaches 29.4% AP / 48.7% AP_{50}, but no val figure is published for UAVDet, so that number is not a like-for-like comparison.)

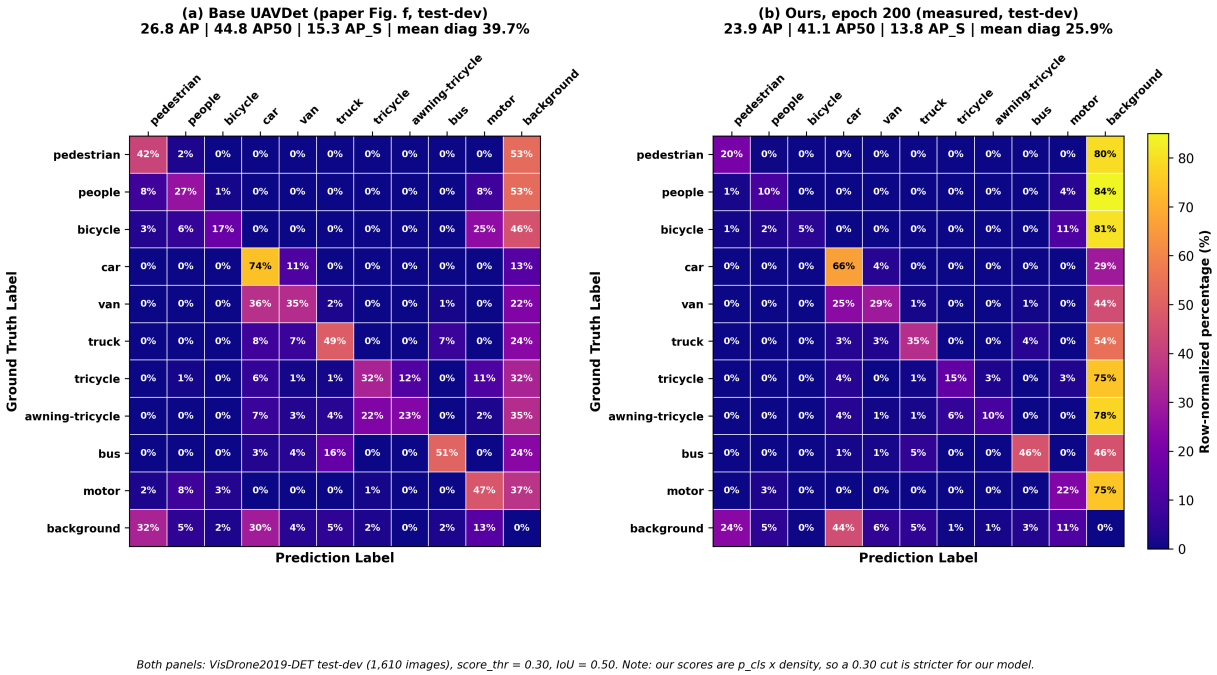


Figure 7: **Normalized confusion matrices on VisDrone2019-DET test-dev (score threshold 0.30, IoU 0.50). (a)** Base UAVDet, transcribed from Fig. (f) of [8] (mean diagonal 39.7%). **(b)** Ours, epoch 200, measured with `tools/deep_eval_visdrone.py` (mean diagonal 25.9%). Each row is normalized over all ground-truth boxes of that class; the last column is the share of ground-truth boxes with no prediction above the threshold.

Three observations follow from the measured matrix:

1. **Lower diagonal, driven by the background column.** Our mean diagonal is 25.9% versus 39.7%. Most of the gap is ground-truth boxes with no detection above 0.30 (last column), e.g. `pedestrian` 80% vs. 53%, `motor` 75% vs. 37%. This agrees with the lower test-dev AP and AP_s: on the comparable split, our model does not yet beat Base UAVDet.

2. **Score calibration is a likely contributor (hypothesis).** Our head outputs $p_{\text{cls}} \times M_{\text{density}}$, a product of two probabilities, so a fixed 0.30 cut is stricter for our model than for a single-probability score. COCO AP does not depend on a threshold, so this cannot explain the AP gap, but it can make the confusion matrix look worse than the ranking quality. Testing this requires re-scoring at a calibrated threshold, which is a different protocol from the paper's figure and must be labelled as such.

3. **Off-diagonal confusions are lower, but this is not evidence on its own.** For example, `van`→`car` is 25% vs. 36% and `bus`→`truck` 5% vs. 16%. Because fewer boxes pass the threshold overall, all cells in a row shrink, so these lower rates cannot be credited to better class separation without a threshold-matched comparison.

On the val split, the switch from Mosaic/MixUp to the plain stage-2 pipeline at epoch 181 was clearly beneficial: logged val AP went from 25.9% (epoch 180) to 26.7% (181), 27.7% (183), 29.1% (194) and 29.4% (200). Thin and rare classes remain the weakest on test-dev: `bicycle` (19.7% AP_{50}), `people` (22.1%), `tricycle` (25.1%) and `awning-tricycle` (25.2%). These are the targets for Phase 2.

References

[1] B. Kang, "PMRNet: Physics-informed multi-scale refinement network for medical image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.

[2] Z. Fu *et al.*, "PET-DINO: Unifying visual cues into Grounding DINO with prompt-enriched training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.

[3] C. Wang *et al.*, "Visual consensus prompting for co-salient object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025.

[4] K. Liao *et al.*, "Multi-prototype compactness and boundary-aware synthesis for unsupervised anomaly detection / Prior-guided boundary localization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.

[5] C. Wang *et al.*, "SOPSeg: Prompt-driven small object instance segmentation in earth observation," in *IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Findings (CVPR-F)*, 2026, arXiv:2509.03002.

[6] Y. Shang *et al.*, "BADANet: Boundary-aware distracted attention network for camouflaged object detection," *IEEE Trans. Artif. Intell. (TAI)*, vol. 7, no. 8, 2026.

[7] X. Yang, T. Li, J. Ling, R. Liu, X. Gu, and H. Zhang, "Asymmetric visual–semantic complementary framework for UAV small object detection (VSCD-UAV)," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, 2026, pp. 2518–2521.

[8] Y. Yang, F. Guo, and P. Niu, "UAVDet: A CNN-Mamba hybrid network for efficient small object detection in UAV imagery," *Comput. Vis. Image Underst. (CVIU)*, vol. 264, p. 104637, 2026.

[9] Z. Wang and C. Ma, "Mamba-YOLO: SSMs-based YOLO for object detection," *arXiv preprint arXiv:2406.05835*, 2024.

[10] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[11] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces (S4)," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[12] Y. Liu *et al.*, "VMamba: Visual state space model," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 103031–103063.

[13] C. Xiao *et al.*, "Spatial-Mamba: Effective visual state space models via structure-aware state fusion," *arXiv preprint arXiv:2410.15091*, 2024.

[14] H. Guo *et al.*, "MambaIRv2: Attentive state space restoration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 28124–28133.

[15] S. Qiao, L.-C. Chen, and A. Yuille, "DetectoRS: Detecting objects with recursive feature pyramid and switchable atrous convolution (SAC)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 10213–10224.

[16] J. Dai *et al.*, "Deformable convolutional networks (DCNv1)," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 764–773.

[17] X. Zhu, H. Hu, S. Lin, and J. Dai, "Deformable ConvNets v2: More deformable, better results," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 9308–9316.

[18] W. Wang *et al.*, "InternImage: Exploring large-scale vision foundation models with deformable convolutions (DCNv3)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 14408–14419.

[19] Y. Chen *et al.*, "Dynamic convolution: Attention over convolution kernels," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11030–11039.

[20] C. Li, A. Zhou, and A. Yao, "Omni-dimensional dynamic convolution (ODConv)," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[21] D. Li *et al.*, "Involution: Inverting the inherence of convolution for visual recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 12321–12330.

[22] H. Su *et al.*, "Pixel-adaptive convolutional neural networks (PAC)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 11166–11175.

[23] H. Gao, X. Zhu, S. Lin, and J. Dai, "Deformable kernels: Adapting effective receptive fields for object deformation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[24] R. Zhang, S. Tang, Y. Zhang, J. Li, and S. Yan, "Scale-adaptive convolutions for scene parsing," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2031–2039.

[25] L. Chen *et al.*, "Frequency-adaptive dilated convolution for semantic segmentation (FADC)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[26] R. Liu *et al.*, "An intriguing failing of convolutional neural networks and the CoordConv solution," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, 2018.

[27] Y. Zhang *et al.*, "UAV-DETR: Efficient end-to-end object detection for unmanned aerial vehicle imagery," *arXiv preprint arXiv:2501.01855*, 2025.

[28] Z. Chen *et al.*, "LGI-DETR: Local-global interaction for UAV object detection," *arXiv preprint arXiv:2503.18785*, 2025.

[29] Z. Zhang, "Drone-YOLO: An efficient neural network method for target detection in drone images," *Drones*, vol. 7, no. 8, p. 526, 2023.

[30] Y. Zhang *et al.*, "FFCA-YOLO for small object detection in remote sensing images," *IEEE Trans. Geosci. Remote Sens. (TGRS)*, vol. 62, 2024.

[31] L. Wang *et al.*, "MFFCI-YOLOv8: A lightweight remote sensing object detection network based on multiscale features fusion and context information," *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens. (J-STARS)*, 2024.

[32] H. Li *et al.*, "LUD-YOLO: An efficient and balanced object detection model for unmanned aerial vehicles," *Inf. Sci.*, 2026.

[33] X. Chen *et al.*, "SL-YOLO: A stronger and lighter drone target detection model," *arXiv preprint*, 2025.

[34] H. Sun *et al.*, "SET: Spectral enhancement for tiny object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025.

[35] X. Cai *et al.*, "Poly kernel inception network for remote sensing detection (PKINet)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[36] X. Cai *et al.*, "PKINet-v2: Towards powerful and efficient poly-kernel remote sensing object detection," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2026, arXiv:2603.16341.

[37] K. Li *et al.*, "Unleashing channel potential: Space-frequency selection convolution for SAR object detection (SFS-Conv)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[38] B. Cao *et al.*, "Visible and clear: Finding tiny objects in difference map (SR-TOD)," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, arXiv:2405.11276.

[39] J. Liu *et al.*, "DeCoF: Tiny object detection in aerial remote sensing imagery via detail-retentive downsampling and conflict-aware fusion," *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens. (J-STARS)*, 2025.

[40] X. Yuan *et al.*, "Small object detection via coarse-to-fine proposal generation and imitation learning (CFINet)," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.

[41] T. Lindeberg, *Scale-Space Theory in Computer Vision*. Springer Science & Business Media, 1994.

[42] P. Perona and J. Malik, "Scale-space and edge detection using anisotropic diffusion," *IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI)*, vol. 12, no. 7, pp. 629–639, 1990.

[43] J. Weickert, "Coherence-enhancing diffusion filtering," *Int. J. Comput. Vis. (IJCV)*, vol. 31, no. 2, pp. 111–127, 1999.

[44] Z. Yang *et al.*, "Structure tensor representation for robust oriented object detection," *arXiv preprint arXiv:2411.10497*, 2024.

[45] M. Lin *et al.*, "VGTok: Variable-granularity tokenization for high-resolution object detection," *arXiv preprint*, 2026.

[46] R. Sunkara and T. Luo, "No more strided convolutions or pooling: A new CNN building block for low-resolution images and small objects (SPD-Conv)," in *Proc. ECML-PKDD*, 2022.

[47] S. Shit *et al.*, "clDice: A novel topology-preserving loss function for tubular structure segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021.

[48] J. Wang, C. Xu, W. Yang, and L. Yu, "A normalized Gaussian Wasserstein distance for tiny object detection (NWD)," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2021, pp. 1194–1200.

[49] C. Xu *et al.*, "RFLA: Gaussian receptive field based label assignment for tiny object detection," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022.

[50] C. Xu *et al.*, "Dot distance for tiny object detection in aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2021, pp. 1192–1201.

[51] D. Liang, W. Xu, Y. Zhu, and Y. Zhou, "Focal inverse distance transform maps for crowd localization (FIDT)," *IEEE Trans. Multimedia (TMM)*, vol. 25, pp. 6040–6052, 2022.

[52] F. Yang, H. Fan, P. Chu, E. Blasch, and H. Ling, "Clustered object detection in aerial images (ClusDet)," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 8311–8320.

[53] C. Li, T. Xu, W. Han, R. Peng, and W. Shen, "Density map guided object detection in aerial images (DMNet)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2020, pp. 190–199.

[54] C. Yang, Z. Huang, and N. Wang, "QueryDet: Cascaded sparse query for accelerating high-resolution small object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022.

[55] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection (SAHI)," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2022, pp. 966–970.

[56] N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, "Soft-NMS — Improving object detection with one line of code," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 5561–5569.

[57] Z. Zheng *et al.*, "Distance-IoU loss: Faster and better learning for bounding box regression (DIoU-NMS)," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 12993–13000.

[58] R. Solovyev, W. Wang, and T. Gabruseva, "Weighted boxes fusion: Ensembling boxes from different object detection models (WBF)," *Image Vis. Comput.*, vol. 107, p. 104117, 2021.

[59] W. Liu *et al.*, "Explicit visual prompting for low-level structure segmentations (EVP)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023.

[60] A. Radford *et al.*, "Learning transferable visual models from natural language supervision (CLIP)," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 8748–8763.

[61] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, "Learning to prompt for vision-language models (CoOp)," *Int. J. Comput. Vis. (IJCV)*, vol. 130, no. 9, pp. 2337–2348, 2022.

[62] D. Du *et al.*, "The unmanned aerial vehicle benchmark: Object detection and tracking (VisDrone)," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 370–386.

[63] P. Zhu *et al.*, "Detection and tracking meet drones challenge," *IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI)*, vol. 44, no. 11, pp. 7380–7399, 2021.

[64] G.-S. Xia *et al.*, "DOTA: A large-scale dataset for object detection in aerial images," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 3974–3983.

[65] J. Ding *et al.*, "Object detection in aerial images: A large-scale benchmark and challenges," *IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI)*, vol. 44, no. 11, pp. 7778–7796, 2021.

[66] Y. Sun, B. Cao, P. Zhu, and Q. Hu, "Drone-based RGB-infrared cross-modality vehicle detection via uncertainty-aware learning," *IEEE Trans. Circuits Syst. Video Technol. (TCSVT)*, vol. 32, no. 10, pp. 6701–6714, 2020.

[67] T.-Y. Lin *et al.*, "Feature pyramid networks for object detection (FPN)," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 2117–2125.

[68] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980–2988.

[69] N. Carion *et al.*, "End-to-end object detection with transformers (DETR)," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 213–229.

[70] X. Zhu *et al.*, "Deformable DETR: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[71] Y. Rao *et al.*, "DynamicViT: Efficient vision transformers with dynamic token sparsification," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021.

[72] Y. Lv *et al.*, "DETRs beat YOLOs on real-time object detection (RT-DETR)," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.